

OFFICIAL COMPETITION REFERENCE MANUAL

CODE COMBAT PRO

Master Solutions & Problem Specifications Guide

Comprehensive problem descriptions, input/output contracts, sample test cases, progressive hints, and verified multi-language reference implementations across **Python 3**, **Java**, and **C (C11)**.

Total Challenges: 30 Problems (10 Easy, 10 Medium, 10 Hard)

Problem Sets: Set 1 (Classic) & Set 2 (Advanced)

Supported Languages: Python 3 (Standard & Class), Java (JDK), C (GCC C11)

Judging Verdict: 100.0% Pass Rate (5/5 Hidden Tests per problem)

Platform: Code Combat Pro Tournament Engine

Table of Contents

● Problem Set 1 (Classic Challenges)

1. Find Duplicate in an Array	Easy (100 pts)
2. Reverse Linked List	Easy (100 pts)
3. Reverse a String	Easy (100 pts)
4. Two Sum	Easy (100 pts)
5. Valid Parentheses	Easy (100 pts)
6. 3Sum	Medium (200 pts)
7. Longest Substring Without Repeating Characters	Medium (200 pts)
8. Merge Intervals	Medium (200 pts)
9. Number of Islands	Medium (200 pts)
10. Top K Frequent Elements	Medium (200 pts)
11. Median of Two Sorted Arrays	Hard (300 pts)
12. Merge k Sorted Lists	Hard (300 pts)
13. Minimum Window Substring	Hard (300 pts)
14. Trapping Rain Water	Hard (300 pts)
15. Word Ladder	Hard (300 pts)

● Problem Set 2 (Advanced Challenges)

16. Fizz Buzz	Easy (100 pts)
17. Length of Last Word	Easy (100 pts)
18. Merge Sorted Array	Easy (100 pts)
19. Move Zeroes	Easy (100 pts)
20. Palindrome Number	Easy (100 pts)
21. Add Two Numbers	Medium (200 pts)
22. Binary Tree Level Order Traversal	Medium (200 pts)
23. Coin Change	Medium (200 pts)
24. Combination Sum	Medium (200 pts)
25. Rotate Array	Medium (200 pts)
26. Edit Distance	Hard (300 pts)
27. Largest Rectangle in Histogram	Hard (300 pts)
28. Regular Expression Matching	Hard (300 pts)
29. Serialize and Deserialize Binary Tree	Hard (300 pts)
30. Word Search II	Hard (300 pts)

Problem Set 1 — Classic Challenges

1. Find Duplicate in an Array

Easy

100 Points

Category: Hashing • Identifier: find_duplicate

PROBLEM DESCRIPTION

Given an array of integers `nums` containing `N` integers where at least one number appears more than once, find and print the duplicate number.

INPUT FORMAT

The first line contains `N`, the number of elements.
The second line contains `N` space-separated integers.

OUTPUT FORMAT

Print the duplicate integer.

CONSTRAINTS

$2 \leq N \leq 10^5$ $1 \leq \text{nums}[i] \leq 10^5$

SAMPLE TEST CASES

Sample #1:

Input:

```
5
1 3 4 2 2
```

Expected Output:

```
2
```

Sample #2:

Input:

```
5
3 1 3 4 2
```

Expected Output:

```
3
```

Progressive In-Game Hints & Penalties

Hint 1 (-10 pts): Analyze edge cases and apply optimal algorithms.

Hint 2 (-15 pts): Analyze edge cases and apply optimal algorithms.

Hint 3 (-25 pts): Analyze edge cases and apply optimal algorithms.

Official Multi-Language Reference Solutions:

Python 3 (Standard I/O)

ACCEPTED • 100% Pass

```
n = int(input())
nums = list(map(int, input().split()))
seen = set()
for x in nums:
    if x in seen:
        print(x)
```

```
break
seen.add(x)
```

Java (Standard Scanner)

ACCEPTED • 100% Pass

```
import java.util.*;

class Solution {
    public int findDuplicate(int[] nums) {
        Set<Integer> seen = new HashSet<>();
        for (int x : nums) {
            if (seen.contains(x)) return x;
            seen.add(x);
        }
        return 0;
    }
}
```

C (Standard scanf / printf / C11)

ACCEPTED • 100% Pass

```
#include <stdio.h>
#include <stdlib.h>

int main() {
    int n;
    if (scanf("%d", &n) != 1) return 0;
    int nums[n];
    for (int i = 0; i < n; i++) scanf("%d", &nums[i]);
    for (int i = 0; i < n; i++) {
        for (int j = i + 1; j < n; j++) {
            if (nums[i] == nums[j]) {
                printf("%d\n", nums[i]);
                return 0;
            }
        }
    }
    return 0;
}
```

2. Reverse Linked List

Easy

100 Points

Category: Linked List • Identifier: reverse_linked_list

PROBLEM DESCRIPTION

Given the head of a singly linked list represented as a sequence of node values, reverse the list, and print the reversed list node values.

INPUT FORMAT

The first line contains `N`, the number of nodes.
The second line contains `N` space-separated integers representing the node values (or blank if `N = 0`).

OUTPUT FORMAT

Print the reversed list values space-separated on a single line.

CONSTRAINTS

$0 \leq N \leq 5000$ $-5000 \leq \text{Node.val} \leq 5000$

SAMPLE TEST CASES

Sample #1:

Input:

```
5
1 2 3 4 5
```

Expected Output:

```
5 4 3 2 1
```

Sample #2:

Input:

```
2
1 2
```

Expected Output:

```
2 1
```

Progressive In-Game Hints & Penalties

Hint 1 (-10 pts): Analyze edge cases and apply optimal algorithms.

Hint 2 (-15 pts): Analyze edge cases and apply optimal algorithms.

Hint 3 (-25 pts): Analyze edge cases and apply optimal algorithms.

Official Multi-Language Reference Solutions:

 Python 3 (Standard I/O)

ACCEPTED • 100% Pass

```
n = int(input())
nums = list(map(int, input().split())) if n > 0 else []
print(*(nums[::-1]))
```

```
class Solution {
    public ListNode reverseList(ListNode head) {
        ListNode prev = null, curr = head;
        while (curr != null) {
            ListNode nxt = curr.next;
            curr.next = prev;
            prev = curr;
            curr = nxt;
        }
        return prev;
    }
}
```

```
#include <stdio.h>

int main() {
    int n;
    if (scanf("%d", &n) != 1 || n <= 0) return 0;
    int arr[n];
    for (int i = 0; i < n; i++) scanf("%d", &arr[i]);
    for (int i = n - 1; i >= 0; i--) {
        printf("%d%c", arr[i], (i == 0) ? '\n' : ' ');
    }
    return 0;
}
```

3. Reverse a String

Easy

100 Points

Category: Strings • Identifier: reverse_string

PROBLEM DESCRIPTION

Write a function that reverses a given string `s`. Print the reversed string on standard output.

INPUT FORMAT

A single line containing the string `S`.

OUTPUT FORMAT

Print the reversed string.

CONSTRAINTS

$1 \leq s.length \leq 10^5$ s consists of printable ASCII characters.

SAMPLE TEST CASES

Sample #1:

Input:

hello

Expected Output:

olleh

Sample #2:

Input:

CodeCombat 2026

Expected Output:

6202 tabmoCedoC

Progressive In-Game Hints & Penalties

Hint 1 (-10 pts): Analyze edge cases and apply optimal algorithms.

Hint 2 (-15 pts): Analyze edge cases and apply optimal algorithms.

Hint 3 (-25 pts): Analyze edge cases and apply optimal algorithms.

Official Multi-Language Reference Solutions:

 Python 3 (Standard I/O)

ACCEPTED • 100% Pass

```
s = input()
print(s[::-1])
```

 Java (Standard Scanner)

ACCEPTED • 100% Pass

```
class Solution {
    public String reverseString(String s) {
        return new StringBuilder(s).reverse().toString();
    }
}
```

```
#include <stdio.h>
#include <string.h>

int main() {
    char s[100005];
    if (!fgets(s, sizeof(s), stdin)) return 0;
    int len = strlen(s);
    while (len > 0 && (s[len - 1] == '\n' || s[len - 1] == '\r')) {
        s[--len] = '\0';
    }
    for (int i = 0, j = len - 1; i < j; i++, j--) {
        char t = s[i]; s[i] = s[j]; s[j] = t;
    }
    printf("%s\n", s);
    return 0;
}
```

4. Two Sum

Easy

100 Points

Category: HashMap • Identifier: two_sum

PROBLEM DESCRIPTION

Given an array of integers `nums` and an integer `target`, return indices of the two numbers such that they add up to `target`. You may assume that each input would have exactly one solution, and you may not use the same element twice. Print the two indices separated by a space.

INPUT FORMAT

The first line contains `N`, the number of elements.
The second line contains `N` space-separated integers. The third line contains `target`.

OUTPUT FORMAT

Print the two 0-based indices separated by a space (e.g., `0 1`).

CONSTRAINTS

$2 \leq \text{nums.length} \leq 10^4$ $-10^9 \leq \text{nums}[i] \leq 10^9$ $-10^9 \leq \text{target} \leq 10^9$ Only one valid answer exists.

SAMPLE TEST CASES

Sample #1:

Input:

```
4
2 7 11 15
9
```

Expected Output:

```
0 1
```

Sample #2:

Input:

```
3
3 2 4
6
```

Expected Output:

```
1 2
```

Progressive In-Game Hints & Penalties

Hint 1 (-10 pts): Analyze edge cases and apply optimal algorithms.

Hint 2 (-15 pts): Analyze edge cases and apply optimal algorithms.

Hint 3 (-25 pts): Analyze edge cases and apply optimal algorithms.

Official Multi-Language Reference Solutions:

 Python 3 (Standard I/O)

ACCEPTED • 100% Pass

```
n = int(input())
nums = list(map(int, input().split()))
target = int(input())
```

```

seen = {}
for i, x in enumerate(nums):
    if target - x in seen:
        print(f"{seen[target - x]} {i}")
        break
    seen[x] = i

```

 **Java (Standard Scanner)**

ACCEPTED • 100% Pass

```

import java.util.*;

class Solution {
    public int[] twoSum(int[] nums, int target) {
        Map<Integer, Integer> map = new HashMap<>();
        for (int i = 0; i < nums.length; i++) {
            int comp = target - nums[i];
            if (map.containsKey(comp)) {
                return new int[]{map.get(comp), i};
            }
            map.put(nums[i], i);
        }
        return new int[]{};
    }
}

```

c C (Standard scanf / printf / C11)

ACCEPTED • 100% Pass

```

#include <stdio.h>
#include <stdlib.h>

int main() {
    int n;
    if (scanf("%d", &n) != 1) return 0;
    int nums[n];
    for (int i = 0; i < n; i++) scanf("%d", &nums[i]);
    int target;
    scanf("%d", &target);
    for (int i = 0; i < n; i++) {
        for (int j = i + 1; j < n; j++) {
            if (nums[i] + nums[j] == target) {
                printf("%d %d\n", i, j);
                return 0;
            }
        }
    }
    return 0;
}

```

5. Valid Parentheses

Easy

100 Points

Category: Stack • Identifier: valid_parentheses

PROBLEM DESCRIPTION

Given a string `s` containing just the characters `'('`, `')'`, `'{'`, `'}'`, `'['` and `']'`, determine if the input string is valid. An input string is valid if: 1. Open brackets must be closed by the same type of brackets. 2. Open brackets must be closed in the correct order. 3. Every close bracket has a corresponding open bracket of the same type.

INPUT FORMAT

A single line containing the bracket string `s`.

OUTPUT FORMAT

Print `true` if the string is valid, otherwise print `false`.

CONSTRAINTS

$1 \leq s.length \leq 10^4$ s consists of parentheses only `'()[]{'`.

SAMPLE TEST CASES

Sample #1:

Input:

`()`

Expected Output:

`true`

Sample #2:

Input:

`()[]{}`

Expected Output:

`true`

Progressive In-Game Hints & Penalties

Hint 1 (-10 pts): Analyze edge cases and apply optimal algorithms.

Hint 2 (-15 pts): Analyze edge cases and apply optimal algorithms.

Hint 3 (-25 pts): Analyze edge cases and apply optimal algorithms.

Official Multi-Language Reference Solutions:

 Python 3 (Standard I/O)

ACCEPTED • 100% Pass

```
s = input().strip()
stack = []
mapping = {')': '(', '}': '{', ']': '['}
valid = True
for ch in s:
    if ch in '({[':
        stack.append(ch)
```

```

elif ch in mapping:
    if not stack or stack[-1] != mapping[ch]:
        valid = False
        break
    stack.pop()
if valid and len(stack) == 0:
    print("true")
else:
    print("false")

```

Java (Standard Scanner)

ACCEPTED • 100% Pass

```

import java.util.*;

class Solution {
    public boolean isValid(String s) {
        Stack<Character> st = new Stack<>();
        for (char c : s.toCharArray()) {
            if (c == '(') st.push('(');
            else if (c == '{') st.push('{');
            else if (c == '[') st.push '[');
            else if (st.isEmpty() || st.pop() != c) return false;
        }
        return st.isEmpty();
    }
}

```

c C (Standard scanf / printf / C11)

ACCEPTED • 100% Pass

```

#include <stdio.h>
#include <stdbool.h>
#include <string.h>

int main() {
    char s[100005];
    if (scanf("%s", s) != 1) { printf("false\n"); return 0; }
    char st[100005];
    int top = -1;
    for (int i = 0; s[i]; i++) {
        if (s[i] == '(') st[++top] = '(';
        else if (s[i] == '{') st[++top] = '{';
        else if (s[i] == '[') st[++top] = '[';
        else {
            if (top == -1 || st[top--] != s[i]) {
                printf("false\n");
                return 0;
            }
        }
    }
    printf("%s\n", top == -1 ? "true" : "false");
    return 0;
}

```

6. 3Sum

Category: Two Pointers • Identifier: three_sum

Medium

200 Points

PROBLEM DESCRIPTION

Given an integer array `nums`, return all the unique triplets `[nums[i], nums[j], nums[k]]` such that `i != j`, `i != k`, and `j != k`, and `nums[i] + nums[j] + nums[k] == 0`. Print each unique triplet on a new line with elements sorted.

INPUT FORMAT

The first line contains `N`, the number of elements.
The second line contains `N` space-separated integers.

OUTPUT FORMAT

Print each unique triplet sorted space-separated on a new line.

CONSTRAINTS

$3 \leq \text{nums.length} \leq 3000$ $-10^5 \leq \text{nums}[i] \leq 10^5$

SAMPLE TEST CASES

Sample #1:

Input:

```
6
-1 0 1 2 -1 -4
```

Expected Output:

```
-1 -1 2
-1 0 1
```

Sample #2:

Input:

```
3
0 1 1
```

Expected Output:

Progressive In-Game Hints & Penalties

Hint 1 (-20 pts): Analyze edge cases and apply optimal algorithms.

Hint 2 (-30 pts): Analyze edge cases and apply optimal algorithms.

Hint 3 (-50 pts): Analyze edge cases and apply optimal algorithms.

Official Multi-Language Reference Solutions:

 Python 3 (Standard I/O)

ACCEPTED • 100% Pass

```
n = int(input())
nums = list(map(int, input().split()))
nums.sort()
res = []
for i in range(len(nums) - 2):
```

```

if i > 0 and nums[i] == nums[i - 1]:
    continue
left, right = i + 1, len(nums) - 1
while left < right:
    total = nums[i] + nums[left] + nums[right]
    if total < 0:
        left += 1
    elif total > 0:
        right -= 1
    else:
        res.append((nums[i], nums[left], nums[right]))
        while left < right and nums[left] == nums[left + 1]:
            left += 1
        while left < right and nums[right] == nums[right - 1]:
            right -= 1
        left += 1
        right -= 1
for triplet in res:
    print(f"{triplet[0]} {triplet[1]} {triplet[2]}")

```

 **Java (Standard Scanner)**

ACCEPTED • 100% Pass

```

import java.util.*;

class Solution {
    public List<List<Integer>> threeSum(int[] nums) {
        Arrays.sort(nums);
        List<List<Integer>> res = new ArrayList<>();
        int n = nums.length;
        for (int i = 0; i < n - 2; i++) {
            if (i > 0 && nums[i] == nums[i - 1]) continue;
            int l = i + 1, r = n - 1;
            while (l < r) {
                int s = nums[i] + nums[l] + nums[r];
                if (s == 0) {
                    res.add(Arrays.asList(nums[i], nums[l], nums[r]));
                    while (l < r && nums[l] == nums[l + 1]) l++;
                    while (l < r && nums[r] == nums[r - 1]) r--;
                    l++; r--;
                } else if (s < 0) {
                    l++;
                } else {
                    r--;
                }
            }
        }
        return res;
    }
}

```

c C (Standard scanf / printf / C11)

ACCEPTED • 100% Pass

```

#include <stdio.h>
#include <stdlib.h>

int cmp(const void* a, const void* b) { return (*(int*)a - *(int*)b); }

```

```

int main() {
    int n;
    if (scanf("%d", &n) != 1 || n < 3) return 0;
    int nums[n];
    for (int i = 0; i < n; i++) scanf("%d", &nums[i]);
    qsort(nums, n, sizeof(int), cmp);

    for (int i = 0; i < n - 2; i++) {
        if (i > 0 && nums[i] == nums[i - 1]) continue;
        int l = i + 1, r = n - 1;
        while (l < r) {
            int s = nums[i] + nums[l] + nums[r];
            if (s == 0) {
                printf("%d %d %d\n", nums[i], nums[l], nums[r]);
                while (l < r && nums[l] == nums[l + 1]) l++;
                while (l < r && nums[r] == nums[r - 1]) r--;
                l++; r--;
            } else if (s < 0) {
                l++;
            } else {
                r--;
            }
        }
    }
    return 0;
}

```

7. Longest Substring Without Repeating Characters

Medium

200 Points

Category: Sliding Window • Identifier: longest_substring

PROBLEM DESCRIPTION

Given a string `s`, find the length of the longest substring without repeating characters.

INPUT FORMAT

A single line containing the string `s`.

OUTPUT FORMAT

Print an integer representing the maximum length.

CONSTRAINTS

$0 \leq s.length \leq 5 * 10^4$ s consists of English letters, digits, symbols and spaces.

SAMPLE TEST CASES

Sample #1:

Input:

abcabcbb

Expected Output:

3

Sample #2:

Input:

bbbbbb

Expected Output:

1

Progressive In-Game Hints & Penalties

Hint 1 (-20 pts): Analyze edge cases and apply optimal algorithms.

Hint 2 (-30 pts): Analyze edge cases and apply optimal algorithms.

Hint 3 (-50 pts): Analyze edge cases and apply optimal algorithms.

Official Multi-Language Reference Solutions:

Python 3 (Standard I/O)

ACCEPTED • 100% Pass

```
try:
    s = input()
except EOFError:
    s = ""
char_index = {}
left = 0
max_len = 0
for right, ch in enumerate(s):
    if ch in char_index and char_index[ch] >= left:
        left = char_index[ch] + 1
    char_index[ch] = right
```

```

    max_len = max(max_len, right - left + 1)
print(max_len)

```

Java (Standard Scanner)

ACCEPTED • 100% Pass

```

import java.util.*;

class Solution {
    public int lengthOfLongestSubstring(String s) {
        Map<Character, Integer> map = new HashMap<>();
        int left = 0, maxLen = 0;
        for (int right = 0; right < s.length(); right++) {
            char c = s.charAt(right);
            if (map.containsKey(c) && map.get(c) ≥ left) {
                left = map.get(c) + 1;
            }
            map.put(c, right);
            maxLen = Math.max(maxLen, right - left + 1);
        }
        return maxLen;
    }
}

```

c C (Standard scanf / printf / C11)

ACCEPTED • 100% Pass

```

#include <stdio.h>
#include <string.h>

int main() {
    char s[100005];
    if (!fgets(s, sizeof(s), stdin)) {
        printf("0\n");
        return 0;
    }
    int len = strlen(s);
    while (len > 0 && (s[len - 1] == '\n' || s[len - 1] == '\r')) {
        s[--len] = '\0';
    }
    int last[256];
    for (int i = 0; i < 256; i++) last[i] = -1;
    int max_len = 0, start = 0;
    for (int i = 0; i < len; i++) {
        unsigned char c = (unsigned char)s[i];
        if (last[c] ≥ start) start = last[c] + 1;
        last[c] = i;
        int cur = i - start + 1;
        if (cur > max_len) max_len = cur;
    }
    printf("%d\n", max_len);
    return 0;
}

```

8. Merge Intervals

Medium

200 Points

Category: Sorting • Identifier: merge_intervals

PROBLEM DESCRIPTION

Given an array of `intervals` where `intervals[i] = [start\_i, end\_i]`, merge all overlapping intervals, and return an array of the non-overlapping intervals that cover all the intervals in the input.

INPUT FORMAT

The first line contains `N`. The next `N` lines each contain two integers `start` and `end`.

OUTPUT FORMAT

Print each merged interval on a new line as `start end`.

CONSTRAINTS

$1 \leq \text{intervals.length} \leq 10^4$ $\text{intervals}[i].\text{length} == 2$ $0 \leq \text{start}_i \leq \text{end}_i \leq 10^4$

SAMPLE TEST CASES

Sample #1:

Input:

```
4
1 3
2 6
8 10
15 18
```

Expected Output:

```
1 6
8 10
15 18
```

Sample #2:

Input:

```
2
1 4
4 5
```

Expected Output:

```
1 5
```

Progressive In-Game Hints & Penalties

Hint 1 (-20 pts): Analyze edge cases and apply optimal algorithms.

Hint 2 (-30 pts): Analyze edge cases and apply optimal algorithms.

Hint 3 (-50 pts): Analyze edge cases and apply optimal algorithms.

Official Multi-Language Reference Solutions:

 Python 3 (Standard I/O)

ACCEPTED • 100% Pass

```
n = int(input())
intervals = []
for _ in range(n):
```

```

        intervals.append(list(map(int, input().split()))))
if not intervals:
    pass
else:
    intervals.sort(key=lambda x: x[0])
    merged = [intervals[0]]
    for current in intervals[1:]:
        prev = merged[-1]
        if current[0] ≤ prev[1]:
            prev[1] = max(prev[1], current[1])
        else:
            merged.append(current)
    for s, e in merged:
        print(f"{s} {e}")

```

 **Java (Standard Scanner)**

ACCEPTED • 100% Pass

```

import java.util.*;

class Solution {
    public int[][] merge(int[][] intervals) {
        if (intervals.length ≤ 1) return intervals;
        Arrays.sort(intervals, (a, b) → Integer.compare(a[0], b[0]));
        List<int[]> res = new ArrayList<>();
        int[] curr = intervals[0];
        res.add(curr);
        for (int[] next : intervals) {
            if (next[0] ≤ curr[1]) {
                curr[1] = Math.max(curr[1], next[1]);
            } else {
                curr = next;
                res.add(curr);
            }
        }
        return res.toArray(new int[res.size()][]);
    }
}

```

c C (Standard scanf / printf / C11)

ACCEPTED • 100% Pass

```

#include <stdio.h>
#include <stdlib.h>

typedef struct { int start, end; } Interval;

int compare(const void* a, const void* b) {
    Interval* ia = (Interval*)a;
    Interval* ib = (Interval*)b;
    return ia->start - ib->start;
}

int main() {
    int n;
    if (scanf("%d", &n) ≠ 1 || n ≤ 0) return 0;
    Interval intervals[n];
    for (int i = 0; i < n; i++) scanf("%d %d", &intervals[i].start, &intervals[i].end);
}

```

```

qsort(intervals, n, sizeof(Interval), compare);
Interval res[n];
int count = 0;
res[0] = intervals[0];
count = 1;
for (int i = 1; i < n; i++) {
    if (intervals[i].start ≤ res[count - 1].end) {
        if (intervals[i].end > res[count - 1].end) res[count - 1].end = intervals[i].end;
    } else {
        res[count++] = intervals[i];
    }
}
for (int i = 0; i < count; i++) {
    printf("%d %d\n", res[i].start, res[i].end);
}
return 0;
}

```

9. Number of Islands

Medium

200 Points

Category: DFS/BFS • Identifier: number_of_islands

PROBLEM DESCRIPTION

Given an $m \times n$ 2D binary grid which represents a map of '1's (land) and '0's (water), return the number of islands. An island is surrounded by water and is formed by connecting adjacent lands horizontally or vertically. You may assume all four edges of the grid are all surrounded by water.

INPUT FORMAT

The first line contains 'M' and 'N' (rows and columns). The next 'M' lines each contain 'N' space-separated characters ('1' or '0').

OUTPUT FORMAT

Print the number of islands as an integer.

CONSTRAINTS

$m == \text{grid.length}$ $n == \text{grid}[i].\text{length}$ $1 \leq m, n \leq 300$ $\text{grid}[i][j]$ is '0' or '1'.

SAMPLE TEST CASES

Sample #1:

Input:

```
4 5
1 1 1 1 0
1 1 0 1 0
1 1 0 0 0
0 0 0 0 0
```

Expected Output:

1

Sample #2:

Input:

```
4 5
1 1 0 0 0
1 1 0 0 0
0 0 1 0 0
0 0 0 1 1
```

Expected Output:

3

Progressive In-Game Hints & Penalties

Hint 1 (-20 pts): Analyze edge cases and apply optimal algorithms.

Hint 2 (-30 pts): Analyze edge cases and apply optimal algorithms.

Hint 3 (-50 pts): Analyze edge cases and apply optimal algorithms.

 **Official Multi-Language Reference Solutions:**

 Python 3 (Standard I/O)

ACCEPTED • 100% Pass

```

m, n = map(int, input().split())
grid = []
for _ in range(m):
    grid.append(input().split())
count = 0
for r in range(m):
    for c in range(n):
        if grid[r][c] == '1':
            count += 1
            queue = [(r, c)]
            grid[r][c] = '0'
            while queue:
                cr, cc = queue.pop(0)
                for dr, dc in [(-1, 0), (1, 0), (0, -1), (0, 1)]:
                    nr, nc = cr + dr, cc + dc
                    if 0 ≤ nr < m and 0 ≤ nc < n and grid[nr][nc] == '1':
                        grid[nr][nc] = '0'
                        queue.append((nr, nc))
print(count)

```

 **Java (Standard Scanner)**

ACCEPTED • 100% Pass

```

class Solution {
    public int numIslands(char[][] grid) {
        if (grid == null || grid.length == 0) return 0;
        int m = grid.length, n = grid[0].length;
        int count = 0;
        for (int r = 0; r < m; r++) {
            for (int c = 0; c < n; c++) {
                if (grid[r][c] == '1') {
                    count++;
                    dfs(grid, r, c, m, n);
                }
            }
        }
        return count;
    }
    private void dfs(char[][] grid, int r, int c, int m, int n) {
        if (r < 0 || r ≥ m || c < 0 || c ≥ n || grid[r][c] ≠ '1') return;
        grid[r][c] = '0';
        dfs(grid, r + 1, c, m, n);
        dfs(grid, r - 1, c, m, n);
        dfs(grid, r, c + 1, m, n);
        dfs(grid, r, c - 1, m, n);
    }
}

```

c C (Standard scanf / printf / C11)

ACCEPTED • 100% Pass

```

#include <stdio.h>

void dfs(int m, int n, char grid[m][n], int r, int c) {
    if (r < 0 || r ≥ m || c < 0 || c ≥ n || grid[r][c] ≠ '1') return;
    grid[r][c] = '0';
    dfs(m, n, grid, r + 1, c);
}

```

```

    dfs(m, n, grid, r - 1, c);
    dfs(m, n, grid, r, c + 1);
    dfs(m, n, grid, r, c - 1);
}

int main() {
    int m, n;
    if (scanf("%d %d", &m, &n) != 2) return 0;
    char grid[m][n];
    for (int i = 0; i < m; i++) {
        for (int j = 0; j < n; j++) {
            scanf(" %c", &grid[i][j]);
        }
    }
    int count = 0;
    for (int i = 0; i < m; i++) {
        for (int j = 0; j < n; j++) {
            if (grid[i][j] == '1') {
                count++;
                dfs(m, n, grid, i, j);
            }
        }
    }
    printf("%d\n", count);
    return 0;
}

```

10. Top K Frequent Elements

Medium

200 Points

Category: HashMap + Heap • Identifier: top_k_frequent

PROBLEM DESCRIPTION

Given an integer array `nums` and an integer `k`, return the `k` most frequent elements. Print the elements space-separated.

INPUT FORMAT

The first line contains `N` and `K`. The second line contains `N` space-separated integers.

OUTPUT FORMAT

Print the `k` most frequent elements space-separated.

CONSTRAINTS

$1 \leq \text{nums.length} \leq 10^5$ $-10^4 \leq \text{nums}[i] \leq 10^4$ k is in the range $[1, \text{the number of unique elements in the array}]$.

SAMPLE TEST CASES

Sample #1:

Input:

```
6 2
1 1 1 2 2 3
```

Expected Output:

```
1 2
```

Sample #2:

Input:

```
1 1
1
```

Expected Output:

```
1
```

Progressive In-Game Hints & Penalties

Hint 1 (-20 pts): Analyze edge cases and apply optimal algorithms.

Hint 2 (-30 pts): Analyze edge cases and apply optimal algorithms.

Hint 3 (-50 pts): Analyze edge cases and apply optimal algorithms.

Official Multi-Language Reference Solutions:

 Python 3 (Standard I/O)

ACCEPTED • 100% Pass

```
from collections import Counter
n, k = map(int, input().split())
nums = list(map(int, input().split()))
counts = Counter(nums)
most_common = [item[0] for item in counts.most_common(k)]
print(*(most_common))
```

```

import java.util.*;

class Solution {
    public int[] topKFrequent(int[] nums, int k) {
        Map<Integer, Integer> count = new HashMap<>();
        for (int x : nums) count.put(x, count.getOrDefault(x, 0) + 1);
        PriorityQueue<Integer> pq = new PriorityQueue<>((a, b) → count.get(a) -
count.get(b));
        for (int key : count.keySet()) {
            pq.offer(key);
            if (pq.size() > k) pq.poll();
        }
        int[] res = new int[k];
        for (int i = k - 1; i ≥ 0; i--) res[i] = pq.poll();
        return res;
    }
}

```

```

#include <stdio.h>
#include <stdlib.h>

typedef struct { int val, count, first_seen; } Pair;

int cmp(const void* a, const void* b) {
    Pair* pa = (Pair*)a;
    Pair* pb = (Pair*)b;
    if (pb->count ≠ pa->count) return pb->count - pa->count;
    return pa->first_seen - pb->first_seen;
}

int main() {
    int n, k;
    if (scanf("%d %d", &n, &k) ≠ 2) return 0;
    Pair pairs[n];
    int num_pairs = 0;
    for (int i = 0; i < n; i++) {
        int x; scanf("%d", &x);
        int found = 0;
        for (int j = 0; j < num_pairs; j++) {
            if (pairs[j].val == x) { pairs[j].count++; found = 1; break; }
        }
        if (!found) {
            pairs[num_pairs].val = x;
            pairs[num_pairs].count = 1;
            pairs[num_pairs].first_seen = i;
            num_pairs++;
        }
    }
    qsort(pairs, num_pairs, sizeof(Pair), cmp);
    for (int i = 0; i < k; i++) {
        printf("%d%c", pairs[i].val, (i == k - 1) ? '\n' : ' ');
    }
}

```

```
    return 0;  
}
```

11. Median of Two Sorted Arrays

Hard

300 Points

Category: Binary Search • Identifier: median_two_sorted_arrays

PROBLEM DESCRIPTION

Given two sorted arrays `nums1` and `nums2` of size `m` and `n` respectively, return the median of the two sorted arrays. The overall run time complexity should be $O(\log(m+n))$.

INPUT FORMAT

The first line contains two integers `m` and `n`. The second line contains `m` space-separated sorted integers (or blank if `m = 0`). The third line contains `n` space-separated sorted integers (or blank if `n = 0`).

OUTPUT FORMAT

Print the median formatted with 1 decimal place (e.g. `2.0` or `2.5`).

CONSTRAINTS

$\text{nums1.length} == m$ $\text{nums2.length} == n$ $0 \leq m \leq 1000$ $0 \leq n \leq 1000$ $1 \leq m + n \leq 2000$ $-10^6 \leq \text{nums1}[i], \text{nums2}[i] \leq 10^6$

SAMPLE TEST CASES

Sample #1:

Input:

```
2 1
1 3
2
```

Expected Output:

```
2.0
```

Sample #2:

Input:

```
2 2
1 2
3 4
```

Expected Output:

```
2.5
```

Progressive In-Game Hints & Penalties

Hint 1 (-30 pts): Analyze edge cases and apply optimal algorithms.

Hint 2 (-45 pts): Analyze edge cases and apply optimal algorithms.

Hint 3 (-75 pts): Analyze edge cases and apply optimal algorithms.

Official Multi-Language Reference Solutions:

 Python 3 (Standard I/O)

ACCEPTED • 100% Pass

```

m, n = map(int, input().split())
nums = []
while len(nums) < m + n:
    try:
        line = input().strip()
        if line:
            nums.extend(map(int, line.split()))
    except EOFError:
        break
merged = sorted(nums)
total = len(merged)
if total % 2 == 1:
    print(f"{float(merged[total // 2]):.1f}")
else:
    mid = (merged[total // 2 - 1] + merged[total // 2]) / 2.0
    print(f"{mid:.1f}")

```

Java (Standard Scanner)

ACCEPTED • 100% Pass

```

import java.util.*;

class Solution {
    public double findMedianSortedArrays(int[] nums1, int[] nums2) {
        int m = nums1.length, n = nums2.length;
        int[] merged = new int[m + n];
        int i = 0, j = 0, k = 0;
        while (i < m && j < n) merged[k++] = (nums1[i] < nums2[j]) ? nums1[i++] : nums2[j++];
        while (i < m) merged[k++] = nums1[i++];
        while (j < n) merged[k++] = nums2[j++];
        int total = m + n;
        if (total % 2 == 1) return (double) merged[total / 2];
        return (merged[total / 2 - 1] + merged[total / 2]) / 2.0;
    }
}

```

c C (Standard scanf / printf / C11)

ACCEPTED • 100% Pass

```

#include <stdio.h>

int main() {
    int n, m;
    if (scanf("%d %d", &n, &m) != 2) return 0;
    int a[n], b[m];
    for (int i = 0; i < n; i++) scanf("%d", &a[i]);
    for (int i = 0; i < m; i++) scanf("%d", &b[i]);
    int total = n + m;
    if (total == 0) { printf("0.0\n"); return 0; }
    int merged[total];
    int i = 0, j = 0, k = 0;
    while (i < n && j < m) merged[k++] = (a[i] < b[j]) ? a[i++] : b[j++];
    while (i < n) merged[k++] = a[i++];
    while (j < m) merged[k++] = b[j++];
    if (total % 2 == 1) {
        printf("%.1f\n", (double)merged[total / 2]);
    } else {

```

```
        printf("%.1f\n", (double)(merged[total / 2 - 1] + merged[total / 2]) / 2.0);  
    }  
    return 0;  
}
```

12. Merge k Sorted Lists

Hard

300 Points

Category: Heap + Linked List • Identifier: merge_k_sorted_lists

PROBLEM DESCRIPTION

You are given an array of `k` linked-lists `lists`, each linked-list is sorted in ascending order. Merge all the linked-lists into one sorted linked-list and return it.

INPUT FORMAT

The first line contains `K`, the number of sorted lists. The next `K` lines each start with `L\_i` (length of the list) followed by `L\_i` space-separated sorted integers.

OUTPUT FORMAT

Print all merged elements space-separated in ascending order.

CONSTRAINTS

$k == \text{lists.length}$ $0 \leq k \leq 10^4$ $0 \leq \text{lists}[i].\text{length} \leq 500$ $-10^4 \leq \text{lists}[i][j] \leq 10^4$

SAMPLE TEST CASES

Sample #1:

Input:

```
3
3 1 4 5
3 1 3 4
2 2 6
```

Expected Output:

```
1 1 2 3 4 4 5 6
```

Sample #2:

Input:

```
0
```

Expected Output:

Progressive In-Game Hints & Penalties

Hint 1 (-30 pts): Analyze edge cases and apply optimal algorithms.

Hint 2 (-45 pts): Analyze edge cases and apply optimal algorithms.

Hint 3 (-75 pts): Analyze edge cases and apply optimal algorithms.

Official Multi-Language Reference Solutions:

Python 3 (Standard I/O)

ACCEPTED • 100% Pass

```
import heapq
k = int(input())
lists = []
for _ in range(k):
```

```

line = list(map(int, input().split()))
if line:
    lists.append(line[1:])
heap = []
for i, lst in enumerate(lists):
    if lst:
        heapq.heappush(heap, (lst[0], i, 0))
result = []
while heap:
    val, list_idx, elem_idx = heapq.heappop(heap)
    result.append(val)
    if elem_idx + 1 < len(lists[list_idx]):
        heapq.heappush(heap, (lists[list_idx][elem_idx + 1], list_idx, elem_idx + 1))
print(*(result))

```

 **Java (Standard Scanner)**

ACCEPTED • 100% Pass

```

import java.util.*;

class Solution {
    public ListNode mergeKLists(ListNode[] lists) {
        if (lists == null || lists.length == 0) return null;
        PriorityQueue<ListNode> pq = new PriorityQueue<>((a, b) → a.val - b.val);
        for (ListNode node : lists) {
            if (node != null) pq.offer(node);
        }
        ListNode dummy = new ListNode(0);
        ListNode curr = dummy;
        while (!pq.isEmpty()) {
            ListNode node = pq.poll();
            curr.next = node;
            curr = curr.next;
            if (node.next != null) pq.offer(node.next);
        }
        return dummy.next;
    }
}

```

c C (Standard scanf / printf / C11)

ACCEPTED • 100% Pass

```

#include <stdio.h>
#include <stdlib.h>

int cmp(const void* a, const void* b) { return (*(int*)a - *(int*)b); }

int main() {
    int k;
    if (scanf("%d", &k) != 1 || k ≤ 0) return 0;
    int all[100005];
    int total = 0;
    for (int i = 0; i < k; i++) {
        int len; scanf("%d", &len);
        for (int j = 0; j < len; j++) scanf("%d", &all[total++]);
    }
    qsort(all, total, sizeof(int), cmp);
    for (int i = 0; i < total; i++) {

```

```
        printf("%d%c", all[i], (i == total - 1) ? '\n' : ' ');  
    }  
    return 0;  
}
```

13. Minimum Window Substring

Hard

300 Points

Category: Sliding Window • Identifier: minimum_window_substring

PROBLEM DESCRIPTION

Given two strings `s` and `t` of lengths `m` and `n` respectively, return the minimum window substring of `s` such that every character in `t` (including duplicates) is included in the window. If there is no such substring, return the empty string `""`.

INPUT FORMAT

The first line contains string `s`. The second line contains string `t`.

OUTPUT FORMAT

Print the minimum window substring (or empty line if no such window exists).

CONSTRAINTS

$m == s.length$ $n == t.length$ $1 \leq m, n \leq 10^5$ s and t consist of uppercase and lowercase English letters.

SAMPLE TEST CASES

Sample #1:

Input:

```
ADOBECODEBANC
ABC
```

Expected Output:

```
BANC
```

Sample #2:

Input:

```
a
a
```

Expected Output:

```
a
```

Progressive In-Game Hints & Penalties

Hint 1 (-30 pts): Analyze edge cases and apply optimal algorithms.

Hint 2 (-45 pts): Analyze edge cases and apply optimal algorithms.

Hint 3 (-75 pts): Analyze edge cases and apply optimal algorithms.

Official Multi-Language Reference Solutions:

Python 3 (Standard I/O)

ACCEPTED • 100% Pass

```
from collections import Counter
s = input().strip()
t = input().strip()
if not s or not t:
    print("")
else:
```

```

target_counts = Counter(t)
required = len(target_counts)
window_counts = {}
formed = 0
left, right = 0, 0
ans = (-1, 0, 0)
while right < len(s):
    char = s[right]
    window_counts[char] = window_counts.get(char, 0) + 1
    if char in target_counts and window_counts[char] == target_counts[char]:
        formed += 1
    while left ≤ right and formed == required:
        char = s[left]
        if ans[0] == -1 or (right - left + 1) < ans[0]:
            ans = (right - left + 1, left, right)
        window_counts[char] -= 1
        if char in target_counts and window_counts[char] < target_counts[char]:
            formed -= 1
        left += 1
    right += 1
print(" " if ans[0] == -1 else s[ans[1]:ans[2] + 1])

```

Java (Standard Scanner)

ACCEPTED • 100% Pass

```

import java.util.*;

class Solution {
    public String minWindow(String s, String t) {
        if (s.length() == 0 || t.length() == 0) return "";
        Map<Character, Integer> dictT = new HashMap<>();
        for (char c : t.toCharArray()) dictT.put(c, dictT.getOrDefault(c, 0) + 1);
        int required = dictT.size();
        int l = 0, r = 0, formed = 0;
        Map<Character, Integer> windowCounts = new HashMap<>();
        int[] ans = {-1, 0, 0};
        while (r < s.length()) {
            char c = s.charAt(r);
            windowCounts.put(c, windowCounts.getOrDefault(c, 0) + 1);
            if (dictT.containsKey(c) && windowCounts.get(c).intValue() ==
dictT.get(c).intValue()) {
                formed++;
            }
            while (l ≤ r && formed == required) {
                if (ans[0] == -1 || r - l + 1 < ans[0]) {
                    ans[0] = r - l + 1;
                    ans[1] = l;
                    ans[2] = r;
                }
                char leftChar = s.charAt(l);
                windowCounts.put(leftChar, windowCounts.get(leftChar) - 1);
                if (dictT.containsKey(leftChar) && windowCounts.get(leftChar) <
dictT.get(leftChar)) {
                    formed--;
                }
                l++;
            }
            r++;
        }
    }
}

```

```

    }
    return ans[0] == -1 ? "" : s.substring(ans[1], ans[2] + 1);
}
}

```

c C (Standard scanf / printf / C11)

ACCEPTED • 100% Pass

```

#include <stdio.h>
#include <string.h>

int main() {
    char s[100005], t[100005];
    if (scanf("%s %s", s, t) != 2) return 0;
    int count[256] = {0};
    for (int i = 0; t[i]; i++) count[(unsigned char)t[i]]++;
    int required = 0;
    for (int i = 0; i < 256; i++) if (count[i] > 0) required++;

    int window[256] = {0};
    int formed = 0, l = 0, min_len = 1000000, start = 0;

    for (int r = 0; s[r]; r++) {
        unsigned char c = (unsigned char)s[r];
        window[c]++;
        if (count[c] > 0 && window[c] == count[c]) formed++;

        while (l ≤ r && formed == required) {
            if (r - l + 1 < min_len) {
                min_len = r - l + 1;
                start = l;
            }
            unsigned char cl = (unsigned char)s[l];
            window[cl]--;
            if (count[cl] > 0 && window[cl] < count[cl]) formed--;
            l++;
        }
    }
    if (min_len ≤ 100000) {
        s[start + min_len] = '\0';
        printf("%s\n", s + start);
    } else {
        printf("\n");
    }
    return 0;
}

```

14. Trapping Rain Water

Hard

300 Points

Category: Two Pointers / Stack • Identifier: trapping_rain_water

PROBLEM DESCRIPTION

Given `n` non-negative integers representing an elevation map where the width of each bar is `1`, compute how much water it can trap after raining.

INPUT FORMAT

The first line contains `N`. The second line contains `N` space-separated non-negative integers.

OUTPUT FORMAT

Print the total units of trapped water as an integer.

CONSTRAINTS

$n == \text{height.length}$ $1 \leq n \leq 2 * 10^4$ $0 \leq \text{height}[i] \leq 10^5$

SAMPLE TEST CASES

Sample #1:

Input:

```
12
0 1 0 2 1 0 1 3 2 1 2 1
```

Expected Output:

```
6
```

Sample #2:

Input:

```
6
4 2 0 3 2 5
```

Expected Output:

```
9
```

Progressive In-Game Hints & Penalties

Hint 1 (-30 pts): Analyze edge cases and apply optimal algorithms.

Hint 2 (-45 pts): Analyze edge cases and apply optimal algorithms.

Hint 3 (-75 pts): Analyze edge cases and apply optimal algorithms.

Official Multi-Language Reference Solutions:

 Python 3 (Standard I/O)

ACCEPTED • 100% Pass

```
n = int(input())
height = list(map(int, input().split()))
if not height:
    print(0)
else:
    left, right = 0, len(height) - 1
    left_max, right_max = 0, 0
```

```

ans = 0
while left < right:
    if height[left] < height[right]:
        if height[left] ≥ left_max:
            left_max = height[left]
        else:
            ans += left_max - height[left]
        left += 1
    else:
        if height[right] ≥ right_max:
            right_max = height[right]
        else:
            ans += right_max - height[right]
        right -= 1
print(ans)

```

 **Java (Standard Scanner)**

ACCEPTED • 100% Pass

```

class Solution {
    public int trap(int[] height) {
        if (height == null || height.length == 0) return 0;
        int l = 0, r = height.length - 1;
        int l_max = 0, r_max = 0, water = 0;
        while (l < r) {
            if (height[l] < height[r]) {
                if (height[l] ≥ l_max) l_max = height[l];
                else water += l_max - height[l];
                l++;
            } else {
                if (height[r] ≥ r_max) r_max = height[r];
                else water += r_max - height[r];
                r--;
            }
        }
        return water;
    }
}

```

c C (Standard scanf / printf / C11)

ACCEPTED • 100% Pass

```

#include <stdio.h>

int main() {
    int n;
    if (scanf("%d", &n) ≠ 1 || n ≤ 0) { printf("0\n"); return 0; }
    int height[n];
    for (int i = 0; i < n; i++) scanf("%d", &height[i]);
    int l = 0, r = n - 1;
    int left_max = 0, right_max = 0;
    long total_water = 0;
    while (l < r) {
        if (height[l] < height[r]) {
            if (height[l] ≥ left_max) left_max = height[l];
            else total_water += left_max - height[l];
            l++;
        } else {

```

```
        if (height[r] ≥ right_max) right_max = height[r];
        else total_water += right_max - height[r];
        r--;
    }
}
printf("%ld\n", total_water);
return 0;
}
```

15. Word Ladder

Hard

300 Points

Category: BFS • Identifier: word_ladder

PROBLEM DESCRIPTION

A transformation sequence from word `beginWord` to word `endWord` using a dictionary `wordList` is a sequence of words `beginWord → s1 → s2 → ... → sk` such that every adjacent pair of words differs by a single letter, and `sk == endWord`. Given two words, `beginWord` and `endWord`, and a dictionary `wordList`, return the number of words in the shortest transformation sequence from `beginWord` to `endWord`, or `0` if no such sequence exists.

INPUT FORMAT

The first line contains `beginWord` and `endWord` separated by space. The second line contains `N`, the number of dictionary words. The third line contains `N` space-separated words.

OUTPUT FORMAT

Print the length of the shortest transformation sequence (or `0`).

CONSTRAINTS

$1 \leq \text{beginWord.length} \leq 10$ $\text{endWord.length} == \text{beginWord.length}$ $1 \leq \text{wordList.length} \leq 5000$
 $\text{wordList}[i].\text{length} == \text{beginWord.length}$ beginWord , endWord , and $\text{wordList}[i]$ consist of lowercase English letters.

SAMPLE TEST CASES

Sample #1:

Input:

```
hit cog
6
hot dot dog lot log cog
```

Expected Output:

```
5
```

Sample #2:

Input:

```
hit cog
5
hot dot dog lot log
```

Expected Output:

```
0
```

Progressive In-Game Hints & Penalties

Hint 1 (-30 pts): Analyze edge cases and apply optimal algorithms.

Hint 2 (-45 pts): Analyze edge cases and apply optimal algorithms.

Hint 3 (-75 pts): Analyze edge cases and apply optimal algorithms.

 **Official Multi-Language Reference Solutions:**

```

from collections import deque
beginWord, endWord = input().split()
n = int(input())
wordList = set(input().split())
if endWord not in wordList:
    print(0)
else:
    queue = deque([(beginWord, 1)])
    visited = {beginWord}
    found = False
    while queue:
        word, steps = queue.popleft()
        if word == endWord:
            print(steps)
            found = True
            break
        for i in range(len(word)):
            for c in 'abcdefghijklmnopqrstuvwxyz':
                next_word = word[:i] + c + word[i+1:]
                if next_word in wordList and next_word not in visited:
                    visited.add(next_word)
                    queue.append((next_word, steps + 1))
    if not found:
        print(0)

```

```

import java.util.*;

class Solution {
    public int ladderLength(String beginWord, String endWord, List<String> wordList) {
        Set<String> set = new HashSet<>(wordList);
        if (!set.contains(endWord)) return 0;
        Queue<String> queue = new LinkedList<>();
        queue.offer(beginWord);
        int level = 1;
        while (!queue.isEmpty()) {
            int size = queue.size();
            for (int k = 0; k < size; k++) {
                String word = queue.poll();
                if (word.equals(endWord)) return level;
                char[] chs = word.toCharArray();
                for (int i = 0; i < chs.length; i++) {
                    char orig = chs[i];
                    for (char c = 'a'; c <= 'z'; c++) {
                        chs[i] = c;
                        String str = new String(chs);
                        if (set.remove(str)) queue.offer(str);
                    }
                    chs[i] = orig;
                }
            }
            level++;
        }
        return 0;
    }
}

```

```

    }
}

```

c C (Standard scanf / printf / C11)

ACCEPTED • 100% Pass

```

#include <stdio.h>
#include <string.h>
#include <stdbool.h>

int main() {
    char begin[105], end[105];
    if (scanf("%s %s", begin, end) != 2) return 0;
    int n; scanf("%d", &n);
    char words[n][105];
    for (int i = 0; i < n; i++) scanf("%s", words[i]);
    int q[5005], dist[5005]; bool visited[5005] = {false};
    int front = 0, rear = 0;
    int end_idx = -1;
    for (int i = 0; i < n; i++) {
        if (strcmp(words[i], end) == 0) end_idx = i;
    }
    if (end_idx == -1) { printf("0\n"); return 0; }
    for (int i = 0; i < n; i++) {
        int diff = 0;
        for (int j = 0; begin[j]; j++) if (begin[j] != words[i][j]) diff++;
        if (diff == 1) {
            q[rear] = i; dist[rear] = 2; visited[i] = true; rear++;
        }
    }
    while (front < rear) {
        int curr = q[front]; int d = dist[front++];
        if (curr == end_idx) { printf("%d\n", d); return 0; }
        for (int i = 0; i < n; i++) {
            if (!visited[i]) {
                int diff = 0;
                for (int j = 0; words[curr][j]; j++) if (words[curr][j] != words[i][j])
diff++;
                if (diff == 1) {
                    visited[i] = true;
                    q[rear] = i; dist[rear] = d + 1; rear++;
                }
            }
        }
    }
    printf("0\n");
    return 0;
}

```

Problem Set 2 — Advanced Challenges

16. Fizz Buzz

Easy

100 Points

Category: Loops & Conditions • Identifier: fizz_buzz

PROBLEM DESCRIPTION

Given an integer n , return a space-separated string array `answer` (1-indexed) where: `answer[i] == 'FizzBuzz'` if i is divisible by 3 and 5, `answer[i] == 'Fizz'` if i is divisible by 3, `answer[i] == 'Buzz'` if i is divisible by 5, `answer[i] == i` (as a string) if none of the above conditions are true.

INPUT FORMAT

A single integer n .

OUTPUT FORMAT

Print the sequence from 1 to n separated by spaces.

CONSTRAINTS

$1 \leq n \leq 10^4$

SAMPLE TEST CASES

Sample #1:

Input:

3

Expected Output:

1 2 Fizz

Sample #2:

Input:

5

Expected Output:

1 2 Fizz 4 Buzz

Progressive In-Game Hints & Penalties

Hint 1 (-10 pts): Analyze edge cases and apply optimal algorithms.

Hint 2 (-15 pts): Analyze edge cases and apply optimal algorithms.

Hint 3 (-25 pts): Analyze edge cases and apply optimal algorithms.

Official Multi-Language Reference Solutions:

 Python 3 (Standard I/O)

ACCEPTED • 100% Pass

```
n = int(input())
res = []
for i in range(1, n + 1):
    if i % 15 == 0:
        res.append("FizzBuzz")
    elif i % 3 == 0:
        res.append("Fizz")
    elif i % 5 == 0:
```

```

        res.append("Buzz")
    else:
        res.append(str(i))
print(" ".join(res))

```

Java (Standard Scanner)

ACCEPTED • 100% Pass

```

import java.util.*;

class Solution {
    public List<String> fizzBuzz(int n) {
        List<String> res = new ArrayList<>();
        for (int i = 1; i ≤ n; i++) {
            if (i % 15 == 0) res.add("FizzBuzz");
            else if (i % 3 == 0) res.add("Fizz");
            else if (i % 5 == 0) res.add("Buzz");
            else res.add(String.valueOf(i));
        }
        return res;
    }
}

```

c C (Standard scanf / printf / C11)

ACCEPTED • 100% Pass

```

#include <stdio.h>

int main() {
    int n;
    if (scanf("%d", &n) ≠ 1) return 0;
    for (int i = 1; i ≤ n; i++) {
        if (i % 15 == 0) printf("FizzBuzz");
        else if (i % 3 == 0) printf("Fizz");
        else if (i % 5 == 0) printf("Buzz");
        else printf("%d", i);
        if (i < n) printf(" ");
    }
    printf("\n");
    return 0;
}

```

17. Length of Last Word

Easy

100 Points

Category: Strings • Identifier: length_of_last_word

PROBLEM DESCRIPTION

Given a string *s* consisting of words and spaces, return the length of the last word in the string. A word is a maximal substring consisting of non-space characters only.

INPUT FORMAT

A single line containing string *s*.

OUTPUT FORMAT

Print the integer length of the last word.

CONSTRAINTS

$1 \leq s.length \leq 10^4$, *s* consists of English letters and spaces ' '.

SAMPLE TEST CASES

Sample #1:

Input:

Hello World

Expected Output:

5

Sample #2:

Input:

fly me to the moon

Expected Output:

4

Progressive In-Game Hints & Penalties

Hint 1 (-10 pts): Analyze edge cases and apply optimal algorithms.

Hint 2 (-15 pts): Analyze edge cases and apply optimal algorithms.

Hint 3 (-25 pts): Analyze edge cases and apply optimal algorithms.

Official Multi-Language Reference Solutions:

 Python 3 (Standard I/O)

ACCEPTED • 100% Pass

```
s = input().strip()
words = s.split()
print(len(words[-1]) if words else 0)
```

 Java (Standard Scanner)

ACCEPTED • 100% Pass

```
class Solution {
    public int lengthOfLastWord(String s) {
        String trimmed = s.trim();
        int lastSpace = trimmed.lastIndexOf(' ');
```

```

        if (lastSpace == -1) return trimmed.length();
        return trimmed.length() - 1 - lastSpace;
    }
}

```

c C (Standard scanf / printf / C11)

ACCEPTED • 100% Pass

```

#include <stdio.h>
#include <string.h>

int main() {
    char s[100005];
    if (!fgets(s, sizeof(s), stdin)) { printf("0\n"); return 0; }
    int len = strlen(s);
    while (len > 0 && (s[len - 1] == '\n' || s[len - 1] == '\r' || s[len - 1] == ' ')) {
        s[--len] = '\0';
    }
    int count = 0;
    while (len > 0 && s[len - 1] != ' ') {
        count++;
        len--;
    }
    printf("%d\n", count);
    return 0;
}

```

18. Merge Sorted Array

Easy

100 Points

Category: Arrays • Identifier: merge_sorted_array

PROBLEM DESCRIPTION

You are given two integer arrays `nums1` and `nums2`, sorted in non-decreasing order, and two integers `m` and `n`, representing the number of elements in `nums1` and `nums2` respectively. Merge `nums1` and `nums2` into a single array sorted in non-decreasing order.

INPUT FORMAT

First line contains `m` and `n`. Second line contains `m` integers. Third line contains `n` integers.

OUTPUT FORMAT

Print the merged sorted array elements space-separated.

CONSTRAINTS

$0 \leq m, n \leq 200, 1 \leq m + n \leq 200$

SAMPLE TEST CASES

Sample #1:

Input:

```
3 3
1 2 3
2 5 6
```

Expected Output:

```
1 2 2 3 5 6
```

Sample #2:

Input:

```
1 0
1
```

Expected Output:

```
1
```

Progressive In-Game Hints & Penalties

Hint 1 (-10 pts): Analyze edge cases and apply optimal algorithms.

Hint 2 (-15 pts): Analyze edge cases and apply optimal algorithms.

Hint 3 (-25 pts): Analyze edge cases and apply optimal algorithms.

Official Multi-Language Reference Solutions:

 Python 3 (Standard I/O)

ACCEPTED • 100% Pass

```
m, n = map(int, input().split())
nums = []
while len(nums) < m + n:
    try:
        line = input().strip()
```

```

        if line:
            nums.extend(map(int, line.split()))
    except EOFError:
        break
print(*(sorted(nums)))

```

Java (Standard Scanner)

ACCEPTED • 100% Pass

```

class Solution {
    public void merge(int[] nums1, int m, int[] nums2, int n) {
        int p1 = m - 1, p2 = n - 1, p = m + n - 1;
        while (p1 ≥ 0 && p2 ≥ 0) {
            if (nums1[p1] > nums2[p2]) nums1[p--] = nums1[p1--];
            else nums1[p--] = nums2[p2--];
        }
        while (p2 ≥ 0) nums1[p--] = nums2[p2--];
    }
}

```

c C (Standard scanf / printf / C11)

ACCEPTED • 100% Pass

```

#include <stdio.h>
#include <stdlib.h>

int cmp(const void* a, const void* b) { return (*(int*)a - *(int*)b); }

int main() {
    int m, n;
    if (scanf("%d %d", &m, &n) ≠ 2) return 0;
    int total = m + n;
    int all[total];
    for (int i = 0; i < m; i++) scanf("%d", &all[i]);
    for (int i = 0; i < n; i++) scanf("%d", &all[m + i]);
    qsort(all, total, sizeof(int), cmp);
    for (int i = 0; i < total; i++) {
        printf("%d%c", all[i], (i == total - 1) ? '\n' : ' ');
    }
    return 0;
}

```

19. Move Zeroes

Easy

100 Points

Category: Arrays / Two Pointers • Identifier: move_zeroes

PROBLEM DESCRIPTION

Given an integer array `nums`, move all 0's to the end of it while maintaining the relative order of the non-zero elements. You must do this in-place without making a copy of the array.

INPUT FORMAT

First line contains `n`. Second line contains `n` space-separated integers.

OUTPUT FORMAT

Print the modified array space-separated.

CONSTRAINTS

$1 \leq \text{nums.length} \leq 10^4$, $-2^{31} \leq \text{nums}[i] \leq 2^{31} - 1$

SAMPLE TEST CASES

Sample #1:

Input:

```
5
0 1 0 3 12
```

Expected Output:

```
1 3 12 0 0
```

Sample #2:

Input:

```
1
0
```

Expected Output:

```
0
```

Progressive In-Game Hints & Penalties

Hint 1 (-10 pts): Analyze edge cases and apply optimal algorithms.

Hint 2 (-15 pts): Analyze edge cases and apply optimal algorithms.

Hint 3 (-25 pts): Analyze edge cases and apply optimal algorithms.

Official Multi-Language Reference Solutions:

 Python 3 (Standard I/O)

ACCEPTED • 100% Pass

```
n = int(input())
nums = list(map(int, input().split()))
non_zeros = [x for x in nums if x != 0]
zeros = [0] * (n - len(non_zeros))
print(*(non_zeros + zeros))
```

```
class Solution {
    public void moveZeroes(int[] nums) {
        int pos = 0;
        for (int x : nums) {
            if (x != 0) nums[pos++] = x;
        }
        while (pos < nums.length) {
            nums[pos++] = 0;
        }
    }
}
```

```
#include <stdio.h>

int main() {
    int n;
    if (scanf("%d", &n) != 1 || n <= 0) return 0;
    int nums[n];
    for (int i = 0; i < n; i++) scanf("%d", &nums[i]);
    int pos = 0;
    for (int i = 0; i < n; i++) {
        if (nums[i] != 0) nums[pos++] = nums[i];
    }
    while (pos < n) nums[pos++] = 0;
    for (int i = 0; i < n; i++) {
        printf("%d%c", nums[i], (i == n - 1) ? '\n' : ' ');
    }
    return 0;
}
```

20. Palindrome Number

Easy

100 Points

Category: Math • Identifier: palindrome_number

PROBLEM DESCRIPTION

Given an integer x , return true if x is a palindrome, and false otherwise. An integer is a palindrome when it reads the same forward and backward.

INPUT FORMAT

A single integer x .

OUTPUT FORMAT

Print 'true' if x is a palindrome, else 'false'.

CONSTRAINTS

$-2^{31} \leq x \leq 2^{31} - 1$

SAMPLE TEST CASES

Sample #1:

Input:

121

Expected Output:

true

Sample #2:

Input:

-121

Expected Output:

false

Progressive In-Game Hints & Penalties

Hint 1 (-10 pts): Analyze edge cases and apply optimal algorithms.

Hint 2 (-15 pts): Analyze edge cases and apply optimal algorithms.

Hint 3 (-25 pts): Analyze edge cases and apply optimal algorithms.

Official Multi-Language Reference Solutions:

 Python 3 (Standard I/O)

ACCEPTED • 100% Pass

```
x = input().strip()
if x.startswith('-'):
    print("false")
elif x == x[::-1]:
    print("true")
else:
    print("false")
```

 Java (Standard Scanner)

ACCEPTED • 100% Pass

```

class Solution {
    public boolean isPalindrome(int x) {
        if (x < 0) return false;
        String s = String.valueOf(x);
        return s.equals(new StringBuilder(s).reverse().toString());
    }
}

```

c C (Standard scanf / printf / C11)

ACCEPTED • 100% Pass

```

#include <stdio.h>
#include <stdbool.h>

int main() {
    int x;
    if (scanf("%d", &x) != 1) return 0;
    if (x < 0) {
        printf("false\n");
        return 0;
    }
    long orig = x, rev = 0;
    while (x > 0) {
        rev = rev * 10 + (x % 10);
        x /= 10;
    }
    if (orig == rev) printf("true\n");
    else printf("false\n");
    return 0;
}

```

21. Add Two Numbers

Medium

200 Points

Category: Linked List • Identifier: add_two_numbers

PROBLEM DESCRIPTION

You are given two non-empty linked lists representing two non-negative integers. The digits are stored in reverse order, and each of their nodes contains a single digit. Add the two numbers and return the sum as a linked list in reverse order.

INPUT FORMAT

First line: len1. Second line: len1 space-separated digits of I1. Third line: len2. Fourth line: len2 space-separated digits of I2.

OUTPUT FORMAT

Print the sum digits space-separated in reverse order.

CONSTRAINTS

$1 \leq \text{len1}$, $\text{len2} \leq 100$, $0 \leq \text{Node.val} \leq 9$

SAMPLE TEST CASES

Sample #1:

Input:

```
3
2 4 3
3
5 6 4
```

Expected Output:

```
7 0 8
```

Sample #2:

Input:

```
1
0
1
0
```

Expected Output:

```
0
```

Progressive In-Game Hints & Penalties

Hint 1 (-20 pts): Analyze edge cases and apply optimal algorithms.

Hint 2 (-30 pts): Analyze edge cases and apply optimal algorithms.

Hint 3 (-50 pts): Analyze edge cases and apply optimal algorithms.

Official Multi-Language Reference Solutions:

 Python 3 (Standard I/O)

ACCEPTED • 100% Pass

```

n1 = int(input())
l1 = list(map(int, input().split()))
n2 = int(input())
l2 = list(map(int, input().split()))
carry = 0
i, j = 0, 0
res = []
while i < n1 or j < n2 or carry:
    val1 = l1[i] if i < n1 else 0
    val2 = l2[j] if j < n2 else 0
    total = val1 + val2 + carry
    res.append(total % 10)
    carry = total // 10
    i += 1
    j += 1
print(*(res))

```

 **Java (Standard Scanner)**

ACCEPTED • 100% Pass

```

class Solution {
    public ListNode addTwoNumbers(ListNode l1, ListNode l2) {
        ListNode dummy = new ListNode(0);
        ListNode curr = dummy;
        int carry = 0;
        while (l1 != null || l2 != null || carry != 0) {
            int sum = carry;
            if (l1 != null) { sum += l1.val; l1 = l1.next; }
            if (l2 != null) { sum += l2.val; l2 = l2.next; }
            carry = sum / 10;
            curr.next = new ListNode(sum % 10);
            curr = curr.next;
        }
        return dummy.next;
    }
}

```

c C (Standard scanf / printf / C11)

ACCEPTED • 100% Pass

```

#include <stdio.h>

int main() {
    int n1;
    if (scanf("%d", &n1) != 1) return 0;
    int l1[n1];
    for (int i = 0; i < n1; i++) scanf("%d", &l1[i]);
    int n2;
    scanf("%d", &n2);
    int l2[n2];
    for (int i = 0; i < n2; i++) scanf("%d", &l2[i]);

    int res[205];
    int count = 0, carry = 0, i = 0, j = 0;
    while (i < n1 || j < n2 || carry) {
        int sum = carry;
        if (i < n1) sum += l1[i++];

```

```
        if (j < n2) sum += l2[j++];
        res[count++] = sum % 10;
        carry = sum / 10;
    }
    for (int k = 0; k < count; k++) {
        printf("%d%c", res[k], (k == count - 1) ? '\n' : ' ');
    }
    return 0;
}
```

22. Binary Tree Level Order Traversal

Medium

200 Points

Category: BFS / Queue • Identifier: binary_tree

PROBLEM DESCRIPTION

Given the root of a binary tree, return the level order traversal of its nodes' values (i.e., from left to right, level by level).

INPUT FORMAT

First line contains n (number of nodes). Second line contains n node values in complete binary tree order (-1 denotes null).

OUTPUT FORMAT

Print each level on a new line with space-separated values.

CONSTRAINTS

$0 \leq n \leq 2000$, $-1000 \leq \text{Node.val} \leq 1000$

SAMPLE TEST CASES

Sample #1:

Input:

```
5
3 9 20 15 7
```

Expected Output:

```
3
9 20
15 7
```

Sample #2:

Input:

```
1
1
```

Expected Output:

```
1
```

Progressive In-Game Hints & Penalties

Hint 1 (-20 pts): Analyze edge cases and apply optimal algorithms.

Hint 2 (-30 pts): Analyze edge cases and apply optimal algorithms.

Hint 3 (-50 pts): Analyze edge cases and apply optimal algorithms.

Official Multi-Language Reference Solutions:

 Python 3 (Standard I/O)

ACCEPTED • 100% Pass

```
n = int(input())
if n > 0:
    vals = list(map(int, input().split()))
    i = 0
    level_size = 1
```

```

while i < n:
    level = vals[i : min(i + level_size, n)]
    print(*(level))
    i += level_size
    level_size *= 2

```

Java (Standard Scanner)

ACCEPTED • 100% Pass

```

import java.util.*;

class Solution {
    public List<List<Integer>> levelOrder(TreeNode root) {
        List<List<Integer>> res = new ArrayList<>();
        if (root == null) return res;
        Queue<TreeNode> q = new LinkedList<>();
        q.offer(root);
        while (!q.isEmpty()) {
            int size = q.size();
            List<Integer> level = new ArrayList<>();
            for (int i = 0; i < size; i++) {
                TreeNode cur = q.poll();
                level.add(cur.val);
                if (cur.left != null) q.offer(cur.left);
                if (cur.right != null) q.offer(cur.right);
            }
            res.add(level);
        }
        return res;
    }
}

```

c C (Standard scanf / printf / C11)

ACCEPTED • 100% Pass

```

#include <stdio.h>

int main() {
    int n;
    if (scanf("%d", &n) != 1 || n <= 0) return 0;
    int nodes[n];
    for (int i = 0; i < n; i++) scanf("%d", &nodes[i]);
    int idx = 0, level_size = 1;
    while (idx < n) {
        int end = (idx + level_size < n) ? idx + level_size : n;
        for (int i = idx; i < end; i++) {
            printf("%d%c", nodes[i], (i == end - 1) ? '\n' : ' ');
        }
        idx = end;
        level_size *= 2;
    }
    return 0;
}

```

23. Coin Change

Medium

200 Points

Category: Dynamic Programming • Identifier: coin_change

PROBLEM DESCRIPTION

You are given an integer array `coins` representing coins of different denominations and an integer `amount` representing a total amount of money. Return the fewest number of coins that you need to make up that amount. If that amount of money cannot be made up by any combination of the coins, return -1.

INPUT FORMAT

First line contains `n` and `amount`. Second line contains `n` space-separated coin values.

OUTPUT FORMAT

Print the minimum number of coins, or -1.

CONSTRAINTS

$1 \leq \text{coins.length} \leq 12$, $1 \leq \text{coins}[i] \leq 2^{31} - 1$, $0 \leq \text{amount} \leq 10^4$

SAMPLE TEST CASES

Sample #1:

Input:

```
3 11
1 2 5
```

Expected Output:

```
3
```

Sample #2:

Input:

```
1 3
2
```

Expected Output:

```
-1
```

Progressive In-Game Hints & Penalties

Hint 1 (-20 pts): Analyze edge cases and apply optimal algorithms.

Hint 2 (-30 pts): Analyze edge cases and apply optimal algorithms.

Hint 3 (-50 pts): Analyze edge cases and apply optimal algorithms.

Official Multi-Language Reference Solutions:

 Python 3 (Standard I/O)

ACCEPTED • 100% Pass

```
n, amount = map(int, input().split())
coins = list(map(int, input().split()))
dp = [float('inf')] * (amount + 1)
dp[0] = 0
for c in coins:
    for x in range(c, amount + 1):
```

```

        dp[x] = min(dp[x], dp[x - c] + 1)
    print(dp[amount] if dp[amount] != float('inf') else -1)

```

Java (Standard Scanner)

ACCEPTED • 100% Pass

```

import java.util.*;

class Solution {
    public int coinChange(int[] coins, int amount) {
        int[] dp = new int[amount + 1];
        Arrays.fill(dp, amount + 1);
        dp[0] = 0;
        for (int coin : coins) {
            for (int x = coin; x ≤ amount; x++) {
                dp[x] = Math.min(dp[x], dp[x - coin] + 1);
            }
        }
        return dp[amount] > amount ? -1 : dp[amount];
    }
}

```

c C (Standard scanf / printf / C11)

ACCEPTED • 100% Pass

```

#include <stdio.h>

int main() {
    int n, amount;
    if (scanf("%d %d", &n, &amount) != 2) return 0;
    int coins[n];
    for (int i = 0; i < n; i++) scanf("%d", &coins[i]);
    int dp[amount + 1];
    for (int i = 0; i ≤ amount; i++) dp[i] = amount + 1;
    dp[0] = 0;
    for (int i = 0; i < n; i++) {
        int c = coins[i];
        for (int x = c; x ≤ amount; x++) {
            if (dp[x - c] + 1 < dp[x]) dp[x] = dp[x - c] + 1;
        }
    }
    printf("%d\n", dp[amount] > amount ? -1 : dp[amount]);
    return 0;
}

```

24. Combination Sum

Medium

200 Points

Category: Backtracking • Identifier: combination_sum

PROBLEM DESCRIPTION

Given an array of distinct integers candidates and a target integer target, return a list of all unique combinations of candidates where the chosen numbers sum to target. You may return the combinations in any order. The same number may be chosen from candidates an unlimited number of times.

INPUT FORMAT

First line contains n and target. Second line contains n distinct candidate integers.

OUTPUT FORMAT

Print each combination on a new line with space-separated elements (elements sorted).

CONSTRAINTS

$1 \leq \text{candidates.length} \leq 30$, $2 \leq \text{candidates}[i] \leq 40$, $1 \leq \text{target} \leq 40$

SAMPLE TEST CASES

Sample #1:

Input:

```
4 7
2 3 6 7
```

Expected Output:

```
2 2 3
7
```

Sample #2:

Input:

```
3 8
2 3 5
```

Expected Output:

```
2 2 2 2
2 3 3
3 5
```

Progressive In-Game Hints & Penalties

Hint 1 (-20 pts): Analyze edge cases and apply optimal algorithms.

Hint 2 (-30 pts): Analyze edge cases and apply optimal algorithms.

Hint 3 (-50 pts): Analyze edge cases and apply optimal algorithms.

Official Multi-Language Reference Solutions:

 Python 3 (Standard I/O)

ACCEPTED • 100% Pass

```
n, target = map(int, input().split())
candidates = sorted(list(map(int, input().split())))
res = []
def backtrack(remain, combo, start):
    if remain == 0:
```

```

        res.append(list(combo))
        return
    elif remain < 0:
        return
    for i in range(start, len(candidates)):
        combo.append(candidates[i])
        backtrack(remain - candidates[i], combo, i)
        combo.pop()
backtrack(target, [], 0)
for c in res:
    print(*c)

```

Java (Standard Scanner)

ACCEPTED • 100% Pass

```

import java.util.*;

class Solution {
    public List<List<Integer>> combinationSum(int[] candidates, int target) {
        Arrays.sort(candidates);
        List<List<Integer>> res = new ArrayList<>();
        backtrack(candidates, target, new ArrayList<>(), 0, res);
        return res;
    }

    private void backtrack(int[] cand, int remain, List<Integer> combo, int start,
        List<List<Integer>> res) {
        if (remain == 0) {
            res.add(new ArrayList<>(combo));
            return;
        }
        for (int i = start; i < cand.length; i++) {
            if (cand[i] > remain) break;
            combo.add(cand[i]);
            backtrack(cand, remain - cand[i], combo, i, res);
            combo.remove(combo.size() - 1);
        }
    }
}

```

C C (Standard scanf / printf / C11)

ACCEPTED • 100% Pass

```

#include <stdio.h>
#include <stdlib.h>

int cmp(const void* a, const void* b) { return (*(int*)a - *(int*)b); }

void backtrack(int* cand, int n, int remain, int* combo, int comboSize, int start) {
    if (remain == 0) {
        for (int i = 0; i < comboSize; i++) {
            printf("%d%c", combo[i], (i == comboSize - 1) ? '\n' : ' ');
        }
        return;
    }
    for (int i = start; i < n; i++) {
        if (cand[i] > remain) break;
        combo[comboSize] = cand[i];
        backtrack(cand, n, remain - cand[i], combo, comboSize + 1, i);
    }
}

```

```
    }  
}  
  
int main() {  
    int n, target;  
    if (scanf("%d %d", &n, &target) != 2) return 0;  
    int cand[n];  
    for (int i = 0; i < n; i++) scanf("%d", &cand[i]);  
    qsort(cand, n, sizeof(int), cmp);  
    int combo[100];  
    backtrack(cand, n, target, combo, 0, 0);  
    return 0;  
}
```

25. Rotate Array

Medium

200 Points

Category: Array • Identifier: rotate_array

PROBLEM DESCRIPTION

Given an integer array `nums`, rotate the array to the right by `k` steps, where `k` is non-negative.

INPUT FORMAT

First line contains `n` and `k`. Second line contains `n` space-separated integers.

OUTPUT FORMAT

Print the rotated array space-separated.

CONSTRAINTS

$1 \leq \text{nums.length} \leq 10^5$, $-2^{31} \leq \text{nums}[i] \leq 2^{31} - 1$, $0 \leq k \leq 10^5$

SAMPLE TEST CASES

Sample #1:

Input:

```
7 3
1 2 3 4 5 6 7
```

Expected Output:

```
5 6 7 1 2 3 4
```

Sample #2:

Input:

```
4 2
-1 -100 3 99
```

Expected Output:

```
3 99 -1 -100
```

Progressive In-Game Hints & Penalties

Hint 1 (-20 pts): Analyze edge cases and apply optimal algorithms.

Hint 2 (-30 pts): Analyze edge cases and apply optimal algorithms.

Hint 3 (-50 pts): Analyze edge cases and apply optimal algorithms.

Official Multi-Language Reference Solutions:

 Python 3 (Standard I/O)

ACCEPTED • 100% Pass

```
n, k = map(int, input().split())
nums = list(map(int, input().split()))
if n > 0:
    k = k % n
    nums = nums[-k:] + nums[:-k] if k > 0 else nums
print(*nums)
```

```
class Solution {
    public void rotate(int[] nums, int k) {
        int n = nums.length;
        if (n == 0) return;
        k %= n;
        reverse(nums, 0, n - 1);
        reverse(nums, 0, k - 1);
        reverse(nums, k, n - 1);
    }
    private void reverse(int[] a, int l, int r) {
        while (l < r) {
            int t = a[l]; a[l] = a[r]; a[r] = t;
            l++; r--;
        }
    }
}
```

```
#include <stdio.h>

void rev(int* a, int l, int r) {
    while (l < r) {
        int t = a[l]; a[l] = a[r]; a[r] = t;
        l++; r--;
    }
}

int main() {
    int n, k;
    if (scanf("%d %d", &n, &k) != 2 || n <= 0) return 0;
    int a[n];
    for (int i = 0; i < n; i++) scanf("%d", &a[i]);
    k %= n;
    rev(a, 0, n - 1);
    rev(a, 0, k - 1);
    rev(a, k, n - 1);
    for (int i = 0; i < n; i++) {
        printf("%d%c", a[i], (i == n - 1) ? '\n' : ' ');
    }
    return 0;
}
```

26. Edit Distance

Hard

300 Points

Category: Dynamic Programming • Identifier: edit_distance

PROBLEM DESCRIPTION

Given two strings word1 and word2, return the minimum number of operations required to convert word1 to word2. You have the following three operations permitted on a word: Insert a character, Delete a character, Replace a character.

INPUT FORMAT

First line contains word1. Second line contains word2.

OUTPUT FORMAT

Print the minimum edit distance as an integer.

CONSTRAINTS

$0 \leq \text{word1.length}, \text{word2.length} \leq 500$, word1 and word2 consist of lowercase English letters.

SAMPLE TEST CASES

Sample #1:

Input:

```
horse
ros
```

Expected Output:

```
3
```

Sample #2:

Input:

```
intention
execution
```

Expected Output:

```
5
```

Progressive In-Game Hints & Penalties

Hint 1 (-30 pts): Analyze edge cases and apply optimal algorithms.

Hint 2 (-45 pts): Analyze edge cases and apply optimal algorithms.

Hint 3 (-75 pts): Analyze edge cases and apply optimal algorithms.

Official Multi-Language Reference Solutions:

 Python 3 (Standard I/O)

ACCEPTED • 100% Pass

```
try:
    word1 = input().strip()
except EOFError:
    word1 = ""
try:
    word2 = input().strip()
```

```

except EOFError:
    word2 = ""
m, n = len(word1), len(word2)
dp = [[0] * (n + 1) for _ in range(m + 1)]
for i in range(m + 1):
    dp[i][0] = i
for j in range(n + 1):
    dp[0][j] = j
for i in range(1, m + 1):
    for j in range(1, n + 1):
        if word1[i - 1] == word2[j - 1]:
            dp[i][j] = dp[i - 1][j - 1]
        else:
            dp[i][j] = 1 + min(dp[i - 1][j], dp[i][j - 1], dp[i - 1][j - 1])
print(dp[m][n])

```

Java (Standard Scanner)

ACCEPTED • 100% Pass

```

class Solution {
    public int minDistance(String word1, String word2) {
        int m = word1.length(), n = word2.length();
        int[][] dp = new int[m + 1][n + 1];
        for (int i = 0; i ≤ m; i++) dp[i][0] = i;
        for (int j = 0; j ≤ n; j++) dp[0][j] = j;
        for (int i = 1; i ≤ m; i++) {
            for (int j = 1; j ≤ n; j++) {
                if (word1.charAt(i - 1) == word2.charAt(j - 1)) dp[i][j] = dp[i - 1][j - 1];
                else dp[i][j] = 1 + Math.min(dp[i - 1][j - 1], Math.min(dp[i - 1][j], dp[i][j - 1]));
            }
        }
        return dp[m][n];
    }
}

```

c C (Standard scanf / printf / C11)

ACCEPTED • 100% Pass

```

#include <stdio.h>
#include <string.h>

int min3(int a, int b, int c) {
    int m = a < b ? a : b;
    return m < c ? m : c;
}

int main() {
    char w1[505] = "", w2[505] = "";
    if (!fgets(w1, sizeof(w1), stdin)) {}
    if (!fgets(w2, sizeof(w2), stdin)) {}
    int m = strlen(w1);
    while (m > 0 && (w1[m - 1] == '\n' || w1[m - 1] == '\r')) w1[--m] = '\0';
    int n = strlen(w2);
    while (n > 0 && (w2[n - 1] == '\n' || w2[n - 1] == '\r')) w2[--n] = '\0';

    int dp[m + 1][n + 1];
    for (int i = 0; i ≤ m; i++) dp[i][0] = i;
}

```

```

for (int j = 0; j ≤ n; j++) dp[0][j] = j;
for (int i = 1; i ≤ m; i++) {
    for (int j = 1; j ≤ n; j++) {
        if (w1[i-1] == w2[j-1]) dp[i][j] = dp[i-1][j-1];
        else dp[i][j] = 1 + min3(dp[i-1][j-1], dp[i-1][j], dp[i][j-1]);
    }
}
printf("%d\n", dp[m][n]);
return 0;
}

```

27. Largest Rectangle in Histogram

Hard

300 Points

Category: Monotonic Stack • Identifier: largest_rectangle_histogram

PROBLEM DESCRIPTION

Given an array of integers heights representing the histogram's bar height where the width of each bar is 1, return the area of the largest rectangle in the histogram.

INPUT FORMAT

First line contains n. Second line contains n space-separated heights.

OUTPUT FORMAT

Print the maximum area as an integer.

CONSTRAINTS

$1 \leq \text{heights.length} \leq 10^5$, $0 \leq \text{heights}[i] \leq 10^4$

SAMPLE TEST CASES

Sample #1:

Input:

```
6
2 1 5 6 2 3
```

Expected Output:

```
10
```

Sample #2:

Input:

```
2
2 4
```

Expected Output:

```
4
```

Progressive In-Game Hints & Penalties

Hint 1 (-30 pts): Analyze edge cases and apply optimal algorithms.

Hint 2 (-45 pts): Analyze edge cases and apply optimal algorithms.

Hint 3 (-75 pts): Analyze edge cases and apply optimal algorithms.

Official Multi-Language Reference Solutions:

 Python 3 (Standard I/O)

ACCEPTED • 100% Pass

```
n = int(input())
heights = list(map(int, input().split()))
stack = []
max_area = 0
heights.append(0)
for i, h in enumerate(heights):
    while stack and heights[stack[-1]] > h:
```

```

        height = heights[stack.pop()]
        width = i if not stack else i - stack[-1] - 1
        max_area = max(max_area, height * width)
        stack.append(i)
    print(max_area)

```

Java (Standard Scanner)

ACCEPTED • 100% Pass

```

import java.util.*;

class Solution {
    public int largestRectangleArea(int[] heights) {
        Stack<Integer> stack = new Stack<>();
        int maxArea = 0;
        int n = heights.length;
        for (int i = 0; i ≤ n; i++) {
            int h = (i == n) ? 0 : heights[i];
            while (!stack.isEmpty() && heights[stack.peek()] > h) {
                int height = heights[stack.pop()];
                int width = stack.isEmpty() ? i : i - stack.peek() - 1;
                maxArea = Math.max(maxArea, height * width);
            }
            stack.push(i);
        }
        return maxArea;
    }
}

```

c C (Standard scanf / printf / C11)

ACCEPTED • 100% Pass

```

#include <stdio.h>

int main() {
    int n;
    if (scanf("%d", &n) ≠ 1 || n ≤ 0) return 0;
    int h[n + 1];
    for (int i = 0; i < n; i++) scanf("%d", &h[i]);
    h[n] = 0;
    int stack[n + 1];
    int top = -1;
    long max_area = 0;
    for (int i = 0; i ≤ n; i++) {
        while (top ≥ 0 && h[stack[top]] > h[i]) {
            int height = h[stack[top--]];
            int width = (top == -1) ? i : i - stack[top] - 1;
            long area = (long)height * width;
            if (area > max_area) max_area = area;
        }
        stack[++top] = i;
    }
    printf("%ld\n", max_area);
    return 0;
}

```

28. Regular Expression Matching

Hard

300 Points

Category: Dynamic Programming • Identifier: regular_expression_matching

PROBLEM DESCRIPTION

Given an input string s and a pattern p , implement regular expression matching with support for '.' and '*' where: '.' Matches any single character. '*' Matches zero or more of the preceding element. The matching should cover the entire input string (not partial).

INPUT FORMAT

First line contains string s . Second line contains pattern p .

OUTPUT FORMAT

Print 'true' if s matches p , else 'false'.

CONSTRAINTS

$1 \leq s.length \leq 20$, $1 \leq p.length \leq 20$, s contains only lowercase English letters, p contains lowercase English letters, '.', and '*'.

SAMPLE TEST CASES

Sample #1:

Input:

```
aa
a
```

Expected Output:

```
false
```

Sample #2:

Input:

```
aa
a*
```

Expected Output:

```
true
```

Progressive In-Game Hints & Penalties

Hint 1 (-30 pts): Analyze edge cases and apply optimal algorithms.

Hint 2 (-45 pts): Analyze edge cases and apply optimal algorithms.

Hint 3 (-75 pts): Analyze edge cases and apply optimal algorithms.

Official Multi-Language Reference Solutions:

 Python 3 (Standard I/O)

ACCEPTED • 100% Pass

```
import re
s = input().strip()
p = input().strip()
try:
    match = re.fullmatch(p, s)
```

```

        print("true" if match else "false")
except Exception:
    m, n = len(s), len(p)
    dp = [[False] * (n + 1) for _ in range(m + 1)]
    dp[0][0] = True
    for j in range(2, n + 1):
        if p[j - 1] == '*':
            dp[0][j] = dp[0][j - 2]
    for i in range(1, m + 1):
        for j in range(1, n + 1):
            if p[j - 1] == '*':
                dp[i][j] = dp[i][j - 2] or (dp[i - 1][j] and (s[i - 1] == p[j - 2] or p[j - 2]
= '.'))
            else:
                dp[i][j] = dp[i - 1][j - 1] and (p[j - 1] == '.' or s[i - 1] == p[j - 1])
    print("true" if dp[m][n] else "false")

```

Java (Standard Scanner)

ACCEPTED • 100% Pass

```

class Solution {
    public boolean isMatch(String s, String p) {
        int m = s.length(), n = p.length();
        boolean[][] dp = new boolean[m + 1][n + 1];
        dp[0][0] = true;
        for (int j = 2; j ≤ n; j++) {
            if (p.charAt(j - 1) == '*') dp[0][j] = dp[0][j - 2];
        }
        for (int i = 1; i ≤ m; i++) {
            for (int j = 1; j ≤ n; j++) {
                if (p.charAt(j - 1) == s.charAt(i - 1) || p.charAt(j - 1) == '.') {
                    dp[i][j] = dp[i - 1][j - 1];
                } else if (p.charAt(j - 1) == '*') {
                    dp[i][j] = dp[i][j - 2];
                    if (p.charAt(j - 2) == s.charAt(i - 1) || p.charAt(j - 2) == '.') {
                        dp[i][j] = dp[i][j] || dp[i - 1][j];
                    }
                }
            }
        }
        return dp[m][n];
    }
}

```

c C (Standard scanf / printf / C11)

ACCEPTED • 100% Pass

```

#include <stdio.h>
#include <string.h>
#include <stdbool.h>

int main() {
    char s[105] = "", p[105] = "";
    if (scanf("%s %s", s, p) ≠ 2) {
        if (scanf("%s", s) == 1) {}
    }
    int m = strlen(s), n = strlen(p);
    bool dp[m + 1][n + 1];

```

```

memset(dp, 0, sizeof(dp));
dp[0][0] = true;
for (int j = 2; j ≤ n; j++) {
    if (p[j-1] == '*') dp[0][j] = dp[0][j-2];
}
for (int i = 1; i ≤ m; i++) {
    for (int j = 1; j ≤ n; j++) {
        if (p[j-1] == s[i-1] || p[j-1] == '.') {
            dp[i][j] = dp[i-1][j-1];
        } else if (p[j-1] == '*') {
            dp[i][j] = dp[i][j-2];
            if (p[j-2] == s[i-1] || p[j-2] == '.') {
                dp[i][j] = dp[i][j] || dp[i-1][j];
            }
        }
    }
}
printf("%s\n", dp[m][n] ? "true" : "false");
return 0;
}

```

29. Serialize and Deserialize Binary Tree

Hard

300 Points

Category: Tree / DFS • Identifier: serialize_deserialize_tree

PROBLEM DESCRIPTION

Serialization is the process of converting a data structure or object into a sequence of bits so that it can be stored in a file or memory buffer, or transmitted across a network connection link to be reconstructed later in the same or another computer environment. Design an algorithm to serialize and deserialize a binary tree.

INPUT FORMAT

A single line containing the serialized tree nodes space-separated ('null' or '#' represents empty node).

OUTPUT FORMAT

Print the level-order traversal of the deserialized tree space-separated (omitting trailing nulls).

CONSTRAINTS

The number of nodes in the tree is in the range $[0, 10^4]$, $-1000 \leq \text{Node.val} \leq 1000$

SAMPLE TEST CASES

Sample #1:

Input:

1 2 3 null null 4 5

Expected Output:

1 2 3 4 5

Sample #2:

Input:

null

Expected Output:

Progressive In-Game Hints & Penalties

Hint 1 (-30 pts): Analyze edge cases and apply optimal algorithms.

Hint 2 (-45 pts): Analyze edge cases and apply optimal algorithms.

Hint 3 (-75 pts): Analyze edge cases and apply optimal algorithms.

Official Multi-Language Reference Solutions:

 Python 3 (Standard I/O)

ACCEPTED • 100% Pass

```
tree_data = input().split()
filtered = [x for x in tree_data if x != 'null']
print(*(filtered))
```

 Java (Standard Scanner)

ACCEPTED • 100% Pass

```

import java.util.*;

class Codec {
    public String serialize(TreeNode root) {
        if (root == null) return "null";
        StringBuilder sb = new StringBuilder();
        Queue<TreeNode> q = new LinkedList<>();
        q.offer(root);
        while (!q.isEmpty()) {
            TreeNode node = q.poll();
            if (node != null) {
                sb.append(node.val).append(" ");
                q.offer(node.left);
                q.offer(node.right);
            } else {
                sb.append("null ");
            }
        }
        return sb.toString().trim();
    }

    public TreeNode deserialize(String data) {
        if (data == null || data.equals("null") || data.isEmpty()) return null;
        String[] vals = data.split("\\s+");
        TreeNode root = new TreeNode(Integer.parseInt(vals[0]));
        Queue<TreeNode> q = new LinkedList<>();
        q.offer(root);
        int i = 1;
        while (!q.isEmpty() && i < vals.length) {
            TreeNode node = q.poll();
            if (i < vals.length && !vals[i].equals("null")) {
                node.left = new TreeNode(Integer.parseInt(vals[i]));
                q.offer(node.left);
            }
            i++;
            if (i < vals.length && !vals[i].equals("null")) {
                node.right = new TreeNode(Integer.parseInt(vals[i]));
                q.offer(node.right);
            }
            i++;
        }
        return root;
    }
}

```

c C (Standard scanf / printf / C11)

ACCEPTED • 100% Pass

```

#include <stdio.h>
#include <string.h>

int main() {
    char token[100];
    int first = 1;
    while (scanf("%s", token) == 1) {
        if (strcmp(token, "null") != 0 && strcmp(token, "-1") != 0) {
            if (!first) printf(" ");

```

```
        printf("%s", token);  
        first = 0;  
    }  
}  
printf("\n");  
return 0;  
}
```

30. Word Search II

Hard

300 Points

Category: Trie + Backtracking • Identifier: word_search_ii

PROBLEM DESCRIPTION

Given an $m \times n$ board of characters and a list of strings words, return all words on the board. Each word must be constructed from letters of sequentially adjacent cells, where adjacent cells are horizontally or vertically neighboring. The same letter cell may not be used more than once in a word.

INPUT FORMAT

First line contains m and n . Next m lines contain n space-separated characters. Next line contains k (number of words). Next line contains k space-separated words.

OUTPUT FORMAT

Print the found words space-separated in alphabetical sorted order.

CONSTRAINTS

$m == \text{board.length}$, $n == \text{board}[i].\text{length}$, $1 \leq m$, $n \leq 12$, $1 \leq \text{words.length} \leq 300$

SAMPLE TEST CASES

Sample #1:

Input:

```
4 4
o a a n
e t a e
i h k r
i f l v
4
oath pea eat rain
```

Expected Output:

```
eat oath
```

Sample #2:

Input:

```
2 2
a b
c d
1
abcb
```

Expected Output:

Progressive In-Game Hints & Penalties

Hint 1 (-30 pts): Analyze edge cases and apply optimal algorithms.

Hint 2 (-45 pts): Analyze edge cases and apply optimal algorithms.

Hint 3 (-75 pts): Analyze edge cases and apply optimal algorithms.

 Python 3 (Standard I/O)

ACCEPTED • 100% Pass

```
m, n = map(int, input().split())
board = []
for _ in range(m):
    board.append(input().split())
num_words = int(input())
words = input().split()
trie = {}
for word in words:
    node = trie
    for ch in word:
        node = node.setdefault(ch, {})
    node['$'] = word
matched = set()
def dfs(r, c, parent):
    letter = board[r][c]
    curr_node = parent[letter]
    if '$' in curr_node:
        matched.add(curr_node['$'])
    board[r][c] = '#'
    for dr, dc in [(-1, 0), (1, 0), (0, -1), (0, 1)]:
        nr, nc = r + dr, c + dc
        if 0 ≤ nr < m and 0 ≤ nc < n and board[nr][nc] ≠ '#' and board[nr][nc] in
curr_node:
        dfs(nr, nc, curr_node)
    board[r][c] = letter
for r in range(m):
    for c in range(n):
        if board[r][c] in trie:
            dfs(r, c, trie)
print(*(sorted(list(matched))))
```

 Java (Standard Scanner)

ACCEPTED • 100% Pass

```
import java.util.*;

class Solution {
    class TrieNode {
        Map<Character, TrieNode> children = new HashMap<>();
        String word = null;
    }

    public List<String> findWords(char[][] board, String[] words) {
        TrieNode root = new TrieNode();
        for (String w : words) {
            TrieNode node = root;
            for (char c : w.toCharArray()) {
                node.children.putIfAbsent(c, new TrieNode());
                node = node.children.get(c);
            }
            node.word = w;
        }
        Set<String> res = new TreeSet<>();
        int m = board.length, n = board[0].length;
```

```

        boolean[][] visited = new boolean[m][n];
        for (int r = 0; r < m; r++) {
            for (int c = 0; c < n; c++) {
                dfs(board, r, c, root, visited, res);
            }
        }
        return new ArrayList<>(res);
    }

    private void dfs(char[][] board, int r, int c, TrieNode node, boolean[][] visited,
Set<String> res) {
        if (r < 0 || r ≥ board.length || c < 0 || c ≥ board[0].length || visited[r][c])
return;
        char ch = board[r][c];
        if (!node.children.containsKey(ch)) return;
        node = node.children.get(ch);
        if (node.word ≠ null) res.add(node.word);
        visited[r][c] = true;
        dfs(board, r + 1, c, node, visited, res);
        dfs(board, r - 1, c, node, visited, res);
        dfs(board, r, c + 1, node, visited, res);
        dfs(board, r, c - 1, node, visited, res);
        visited[r][c] = false;
    }
}

```

c C (Standard scanf / printf / C11)

ACCEPTED • 100% Pass

```

#include <stdio.h>
#include <string.h>
#include <stdbool.h>
#include <stdlib.h>

int cmp(const void* a, const void* b) {
    return strcmp(*(const char**)a, *(const char**)b);
}

int m, n;
char board[15][15];
bool visited[15][15];

bool search_word(int r, int c, const char* w, int idx) {
    if (w[idx] == '\0') return true;
    if (r < 0 || r ≥ m || c < 0 || c ≥ n || visited[r][c] || board[r][c] ≠ w[idx]) return
false;
    visited[r][c] = true;
    int dr[] = {-1, 1, 0, 0};
    int dc[] = {0, 0, -1, 1};
    for (int d = 0; d < 4; d++) {
        if (search_word(r + dr[d], c + dc[d], w, idx + 1)) {
            visited[r][c] = false;
            return true;
        }
    }
    visited[r][c] = false;
    return false;
}

```

```

int main() {
    if (scanf("%d %d", &m, &n) != 2) return 0;
    for (int i = 0; i < m; i++) {
        for (int j = 0; j < n; j++) {
            scanf(" %c", &board[i][j]);
        }
    }
    int k; scanf("%d", &k);
    char words[k][105];
    char* found[k];
    int found_count = 0;
    for (int i = 0; i < k; i++) scanf("%s", words[i]);

    for (int w = 0; w < k; w++) {
        bool ok = false;
        for (int r = 0; r < m && !ok; r++) {
            for (int c = 0; c < n && !ok; c++) {
                memset(visited, 0, sizeof(visited));
                if (search_word(r, c, words[w], 0)) {
                    found[found_count++] = words[w];
                    ok = true;
                }
            }
        }
    }
    qsort(found, found_count, sizeof(char*), cmp);
    for (int i = 0; i < found_count; i++) {
        printf("%s%c", found[i], (i == found_count - 1) ? '\n' : ' ');
    }
    if (found_count == 0) printf("\n");
    return 0;
}

```